Agent configurations:

```
from google.adk.models.lite_llm import LiteLlm
from app.core.config import settings

# ── TIER 1: Research Agent + Orchestrator ──────────────────────────
# Primary:  gemini-3.1-pro-preview (Google AI Studio)
# Failover: gemini-1.5-pro (Google AI Studio)
TIER_1_MODEL = LiteLlm(
    model=settings.GEMINI_TIER1_MODEL,          # e.g. "gemini-3.1-pro-preview"
    fallbacks=[settings.GEMINI_TIER1_FALLBACK], # e.g. "gemini/gemini-1.5-pro"
    # GOOGLE_API_KEY is read automatically from the environment
)

# ── TIER 2: 6 SDLC Technical Agents ────────────────────────────────
# Primary:  gemini-3.5-flash (Google AI Studio)
# Failover: gemini-2.0-flash (Google AI Studio)
TIER_2_MODEL = LiteLlm(
    model=settings.GEMINI_TIER2_MODEL,          # e.g. "gemini/gemini-3.5-flash"
    fallbacks=[settings.GEMINI_TIER2_FALLBACK], # e.g. "gemini/gemini-2.0-flash"
    # GOOGLE_API_KEY is read automatically from the environment
)

# ── Context retention — shared generation config ──────────────────
# Ensures full conversation history is preserved across agent turns
GENERATION_CONFIG = {
    "temperature": 0.2,
    "top_p": 0.95,
    "max_output_tokens": 8192,
}

# ── Session config — prevents context loss between turns ──────────
SESSION_CONFIG = {
    "persist_history": True,       # retain full conversation history
    "max_history_turns": 50,       # keep last 50 turns in context window
    "include_system_prompt": True, # always prepend system prompt
}

# agents/research_agent.py

from google.adk.agents import Agent
```

```
from google.adk.tools import FunctionTool, google_search
from agents.models import TIER_1_MODEL, SESSION_CONFIG, GENERATION_CONFIG
from config.prompts import RESEARCH_AGENT_PROMPT
from tools.web import fetch_url, detect_domain_compliance
from tools.storage import save_intermediate
from tools.knowledge_base import rag_query          # now uses local/S3 RAG, not Bedrock KB
from services.dynamodb import update_project_status

research_agent = Agent(
    name="digitano_research_agent",
    model=TIER_1_MODEL,                          # gemini/gemini-3.1-pro-preview
    description=(
        "Senior Research Analyst specialising in software product discovery. "
        "Transforms vague app ideas into structured research context and "
        "generates domain-specific questionnaires for each SDLC specialist."
    ),
    instruction=RESEARCH_AGENT_PROMPT,
    # ── Context retention
──────────────────────────────────────────────────────────────────────────────

    # Pass SESSION_CONFIG so full conversation history is preserved
    # across every turn — no context is lost between agent calls.
    generation_config=GENERATION_CONFIG,
    session_config=SESSION_CONFIG,
    tools=[
        google_search,                          # Native Google ADK search tool
        FunctionTool(fetch_url),                  # Real URL fetcher
        FunctionTool(detect_domain_compliance),     # Compliance signal detection
        FunctionTool(save_intermediate),           # Persist to S3
        FunctionTool(rag_query),                  # Local/S3 RAG (Bedrock KB removed)
        FunctionTool(update_project_status),        # DynamoDB status tracking
    ]
)

# agents/orchestrator.py

from google.adk.agents import Agent
from google.adk.tools import FunctionTool
from agents.models import TIER_1_MODEL, SESSION_CONFIG, GENERATION_CONFIG
from config.prompts import ORCHESTRATOR_PROMPT
from tools.storage import save_artifact, read_intermediate
from tools.knowledge_base import rag_query          # now uses local/S3 RAG, not Bedrock KB
from tools.quality import audit_banned_tokens, validate_gherkin
from services.dynamodb import update_project_status
```

```
# Sub-agents imported so the Orchestrator can delegate to them
from agents.team.product_owner import product_owner_agent
from agents.team.solutions_architect import solutions_architect_agent
from agents.team.database_engineer import database_engineer_agent
from agents.team.fullstack_engineer import fullstack_engineer_agent
from agents.team.backend_engineer import backend_engineer_agent
from agents.team.software_analyst import software_analyst_agent

orchestrator_agent = Agent(
    name="digitano_orchestrator",
    model=TIER_1_MODEL,                    # gemini/gemini-3.1-pro-preview
    description=(
        "Senior Engineering Lead and Documentation Architect. "
        "Coordinates the technical team, resolves conflicts between agent "
        "responses, and assembles the 5-part artifact documentation suite."
    ),
    instruction=ORCHESTRATOR_PROMPT,
    # ── Context retention ──────────────────────────────────────────────
    # SESSION_CONFIG ensures the orchestrator carries full history when
    # delegating to sub-agents — no context dropped on handoff.
    generation_config=GENERATION_CONFIG,
    session_config=SESSION_CONFIG,
    sub_agents=[
        product_owner_agent,
        solutions_architect_agent,
        database_engineer_agent,
        fullstack_engineer_agent,
        backend_engineer_agent,
        software_analyst_agent,
    ],
    tools=[
        FunctionTool(save_artifact),
        FunctionTool(read_intermediate),
        FunctionTool(rag_query),
        FunctionTool(audit_banned_tokens),
        FunctionTool(validate_gherkin),
        FunctionTool(update_project_status),
    ]
)


# agents/team/ — all 6 SDLC technical agents
```

```
# Each agent uses TIER_2_MODEL (gemini/gemini-3.5-flash → gemini/gemini-2.0-flash
fallback)
# SESSION_CONFIG is applied to every agent to prevent context loss.
# Prompts live in config/prompts.py — edit behaviour there.

from google.adk.agents import Agent
from google.adk.tools import FunctionTool
from agents.models import TIER_2_MODEL, SESSION_CONFIG, GENERATION_CONFIG
from tools.storage import save_intermediate, read_intermediate
from tools.knowledge_base import rag_query          # local/S3 RAG, not Bedrock KB
from config.prompts import (
    PRODUCT_OWNER_PROMPT,
    SOLUTIONS_ARCHITECT_PROMPT,
    DATABASE_ENGINEER_PROMPT,
    FULLSTACK_ENGINEER_PROMPT,
    BACKEND_ENGINEER_PROMPT,
    SOFTWARE_ANALYST_PROMPT,
)

# Shared tool set for all 6 technical agents
_TEAM_TOOLS = [
    FunctionTool(rag_query),
    FunctionTool(read_intermediate),
    FunctionTool(save_intermediate),
]

# Shared config applied to every team agent for context retention
_AGENT_KWARGS = dict(
    model=TIER_2_MODEL,                         # gemini/gemini-3.5-flash
    generation_config=GENERATION_CONFIG,
    session_config=SESSION_CONFIG,
    tools=_TEAM_TOOLS,
)

# ── product_owner.py ──────────────────────────────────────────────────────────

product_owner_agent = Agent(
    name="product_owner",
    description="Product Owner - user stories, acceptance criteria, MVP scope",
    instruction=PRODUCT_OWNER_PROMPT,
    **_AGENT_KWARGS
)
```

```
# ── solutions_architect.py ──────────────────────────────────────────────────
solutions_architect_agent = Agent(
    name="solutions_architect",
    description="Solutions Architect - system design, AWS services, integrations",
    instruction=SOLUTIONS_ARCHITECT_PROMPT,
    **_AGENT_KWARGS
)

# ── database_engineer.py ────────────────────────────────────────────────────
database_engineer_agent = Agent(
    name="database_engineer",
    description="Database Engineer - schema design, entities, relationships",
    instruction=DATABASE_ENGINEER_PROMPT,
    **_AGENT_KWARGS
)

# ── fullstack_engineer.py ───────────────────────────────────────────────────
fullstack_engineer_agent = Agent(
    name="fullstack_engineer",
    description="Full-Stack Engineer - UI/UX, frontend stack, component design",
    instruction=FULLSTACK_ENGINEER_PROMPT,
    **_AGENT_KWARGS
)

# ── backend_engineer.py ─────────────────────────────────────────────────────
backend_engineer_agent = Agent(
    name="backend_engineer",
    description="Backend Engineer - API design, business logic, auth, jobs",
    instruction=BACKEND_ENGINEER_PROMPT,
    **_AGENT_KWARGS
)

# ── software_analyst.py ─────────────────────────────────────────────────────
software_analyst_agent = Agent(
    name="software_analyst",
    description="Software Analyst - NFRs, compliance, risk, testing strategy",
    instruction=SOFTWARE_ANALYST_PROMPT,
    **_AGENT_KWARGS
)
```

```
# pipeline/runner.py

# Orchestrates the full pipeline:
#   1. Research Agent  (sequential - must complete first)
#   2. 6 SDLC Agents   (parallel - all run simultaneously)
#   3. Orchestrator    (sequential - assembles final artifacts)

import asyncio
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

from agents.research_agent import research_agent
from agents.orchestrator import orchestrator_agent
from agents.team.product_owner import product_owner_agent
from agents.team.solutions_architect import solutions_architect_agent
from agents.team.database_engineer import database_engineer_agent
from agents.team.fullstack_engineer import fullstack_engineer_agent
from agents.team.backend_engineer import backend_engineer_agent
from agents.team.software_analyst import software_analyst_agent
from services.dynamodb import update_project_status
from tools.storage import read_intermediate

SDLC_AGENTS = [
    product_owner_agent,
    solutions_architect_agent,
    database_engineer_agent,
    fullstack_engineer_agent,
    backend_engineer_agent,
    software_analyst_agent,
]


async def _run_agent(agent, prompt: str, session_service) -> str:
    """Runs a single ADK agent and returns its final text response."""
    runner = Runner(
        agent=agent,
        app_name="digitano",
        session_service=session_service,
    )
    session = await session_service.create_session(
        app_name="digitano",
        user_id="pipeline",
```

```
    )
    response_text = ""
    async for event in runner.run_async(
        user_id="pipeline",
        session_id=session.id,
        new_message=Content(parts=[Part(text=prompt)]),
    ):
        if event.is_final_response() and event.content:
            for part in event.content.parts:
                if part.text:
                    response_text += part.text
    return response_text


async def run_pipeline(project_id: str, user_email: str,
                       user_prompt: str) -> dict:
    """
    Full pipeline entry point. Called by POST /api/generate-prd.
    Returns {project_id, status, artifacts} on success.
    """
    session_service = InMemorySessionService()

    # ---------------------------------------------------------------- #
    # STAGE 1 — Research Agent                                         #
    # ---------------------------------------------------------------- #
    await update_project_status(user_email, project_id, "RESEARCH_IN_PROGRESS")

    research_prompt = (
        f"Project ID: {project_id}\n"
        f"User brief: {user_prompt}\n\n"
        "Research this idea thoroughly and produce research_context "
        "plus 6 tailored questionnaires as described in your instructions."
    )
    await _run_agent(research_agent, research_prompt, session_service)
    await update_project_status(user_email, project_id, "RESEARCH_COMPLETE")

    # Load research output saved to S3 by the Research Agent
    research_context  = await asyncio.to_thread(
        read_intermediate, project_id, "research_context.json"
    )
    questionnaires    = await asyncio.to_thread(
        read_intermediate, project_id, "questionnaires.json"
    )
```

```
# ---------------------------------------------------------------- #
# STAGE 2 — 6 SDLC Agents in parallel                              #
# ---------------------------------------------------------------- #
await update_project_status(user_email, project_id, "IN_PROGRESS")

agent_names = [
    "product_owner", "solutions_architect", "database_engineer",
    "fullstack_engineer", "backend_engineer", "software_analyst",
]

async def run_sdlc_agent(agent, name: str) -> tuple:
    questionnaire = questionnaires.get(name, {})
    prompt = (
        f"Project ID: {project_id}\n"
        f"Research context:\n{research_context}\n\n"
        f"Your questionnaire:\n{questionnaire}\n\n"
        "Answer every question in your questionnaire using the research "
        "context. Return structured JSON as described in your instructions."
    )
    result = await _run_agent(agent, prompt, session_service)
    return name, result

tasks = [
    run_sdlc_agent(agent, name)
    for agent, name in zip(SDLC_AGENTS, agent_names)
]
results = await asyncio.gather(*tasks)
agent_responses = dict(results)

# ---------------------------------------------------------------- #
# STAGE 3 — Orchestrator assembles 5 artifacts                     #
# ---------------------------------------------------------------- #
orchestrator_prompt = (
    f"Project ID: {project_id}\n"
    f"User email: {user_email}\n\n"
    f"Research context:\n{research_context}\n\n"
    f"Agent responses:\n{agent_responses}\n\n"
    "Resolve conflicts, assemble all 5 artifacts, run the quality gate, "
    "save artifacts to S3, and mark the project COMPLETE."
)
await _run_agent(orchestrator_agent, orchestrator_prompt, session_service)

return {"project_id": project_id, "status": "COMPLETE"}
```

```
# pipeline/assembly.py
# 5 artifact assembly functions.
# Called by the Orchestrator agent via save_artifact (tools/storage.py).
# Each function takes the relevant agent responses and returns Markdown.


def assemble_research_document(research_context: dict,
                               agent_responses: dict) -> str:
    rc = research_context
    return f"""# Research Document

## Product Summary
{rc.get('product_summary', '')}

## Target Users
{rc.get('target_users', '')}

## Core Problems
{chr(10).join(f'- {p}' for p in rc.get('core_problems', []))}

## Key Features
{chr(10).join(f'- {f}' for f in rc.get('key_features', []))}

## Competitive Landscape
{chr(10).join(f'- {c}' for c in rc.get('competitive_landscape', []))}

## Recommended Tech Signals
{rc.get('recommended_tech_signals', '')}

## Compliance Signals
{rc.get('compliance_signals', '')}

## Open Questions
{chr(10).join(f'- {q}' for q in rc.get('open_questions', []))}

## Agent Responses
{chr(10).join(f'### {k}{chr(10)}{v}' for k, v in agent_responses.items())}
"""


def assemble_prd(product_owner: dict, software_analyst: dict) -> str:
    return f"""# Product Requirements Document

## User Personas
```

```
{product_owner.get('user_personas', '')}

## User Stories
{product_owner.get('user_stories', '')}

## MVP Features (MoSCoW)
{product_owner.get('mvp_features', '')}

## Out of Scope
{product_owner.get('out_of_scope', '')}

## Success Metrics
{product_owner.get('success_metrics', '')}

## Non-Functional Requirements
{software_analyst.get('non_functional_requirements', '')}

## Compliance Considerations
{software_analyst.get('compliance_considerations', '')}

## Risk Register
{software_analyst.get('risk_register', '')}

## Testing Strategy
{software_analyst.get('testing_strategy', '')}

## Definition of Done
{software_analyst.get('definition_of_done', '')}
"""


def assemble_database_schema(database_engineer: dict,
                             solutions_architect: dict) -> str:
    return f"""# Database Schema

## Recommended Database
{database_engineer.get('recommended_db', '')}

## Entities
{database_engineer.get('entities', '')}

## Relationships
{database_engineer.get('relationships', '')}
```

```
## Indexes
{database_engineer.get('indexes', '')}

## Data Patterns
{database_engineer.get('data_patterns', '')}

## Migration Strategy
{database_engineer.get('migration_strategy', '')}

## Entity Relationship Diagram
```mermaid
{database_engineer.get('mermaid_erd', '')}
```

Architecture Context
{solutions_architect.get('architecture_pattern', '')}
"""

def assemble_api_document(backend_engineer: dict,
                          solutions_architect: dict) -> str:
    return f"""# API Document

API Style
{backend_engineer.get('api_style', '')}

Authentication Strategy
{backend_engineer.get('auth_strategy', '')}

Endpoints
{backend_engineer.get('endpoints', '')}

Business Logic Modules
{backend_engineer.get('business_logic_modules', '')}

Background Jobs
{backend_engineer.get('background_jobs', '')}

Error Handling
{backend_engineer.get('error_handling', '')}

Rate Limiting
{backend_engineer.get('rate_limiting', '')}
```

```
## AWS Services
{solutions_architect.get('aws_services', '')}
"""


def assemble_vibe_code_prompts(fullstack_engineer: dict,
                               prd: str, db_schema: str,
                               api_doc: str) -> str:
    return f"""# Vibe Code Prompts

## Frontend Framework
{fullstack_engineer.get('frontend_framework', '')}

## UI Components
{fullstack_engineer.get('ui_components', '')}

## Page Structure
{fullstack_engineer.get('page_structure', '')}

## State Management
{fullstack_engineer.get('state_management', '')}

## UX Patterns
{fullstack_engineer.get('ux_patterns', '')}

## Vibe Code Hints
{fullstack_engineer.get('vibe_code_hints', '')}

---
## Reference: PRD Summary
{prd[:2000]}

## Reference: Database Schema Summary
{db_schema[:2000]}

## Reference: API Document Summary
{api_doc[:2000]}
"""


# config/prompts.py
# Edit agent behaviour here — no need to touch agent definition files.

RESEARCH_AGENT_PROMPT = """
You are the Research Agent for Digitano Dev-Docs — a Senior Research Analyst
```

specialising in software product discovery. You are the FIRST agent in the
pipeline. You receive a plain-language app idea from a non-technical user.

STEP 1 — WEB RESEARCH
Use google_search to research: existing similar products, common technical
patterns, user pain points, compliance considerations, popular tech stacks.
Perform 3-5 targeted searches. Use fetch_url to read key pages in full.
Only fetch URLs returned by google_search — never arbitrary user-supplied URLs.

STEP 2 — KNOWLEDGE BASE
Use rag_query at least 3 times with different angles to retrieve relevant
SDLC patterns, API design standards, and schema best practices.

STEP 3 — ENRICH THE BRIEF
Produce research_context = {
  product_summary, target_users, core_problems, key_features,
  competitive_landscape, recommended_tech_signals,
  compliance_signals, open_questions, domain_tags
}

STEP 4 — GENERATE 6 QUESTIONNAIRES
One tailored questionnaire per technical agent:
product_owner, solutions_architect, database_engineer,
fullstack_engineer, backend_engineer, software_analyst.
Each: 5-8 questions + context_summary + output_format_hint.

STEP 5 — PERSIST
Use save_intermediate to save research_context and questionnaires to S3.
Return JSON: { research_context, questionnaires }
"""

ORCHESTRATOR_PROMPT = """
You are the Orchestrator for Digitano Dev-Docs — Senior Engineering Lead.
You receive research_context and all 6 SDLC agent responses.
Sub-agents do NOT talk to each other — you are the single source of truth.

STEP 1 — CONFLICT RESOLUTION
Scan all 6 responses for contradictions, gaps, naming inconsistencies.
Use rag_query to validate decisions against SDLC best practices.

STEP 2 — ASSEMBLE 5 ARTIFACTS (in order)
1. Research Document — research_context + all agent responses
2. PRD   — product_owner + software_analyst
3. Database Schema   — database_engineer + solutions_architect

4. API Document — backend_engineer + solutions_architect
5. Vibe Code Prompts — fullstack_engineer + ALL prior artifacts
Save each as Markdown via save_artifact.

STEP 3 — QUALITY GATE
Run audit_banned_tokens and validate_gherkin on all 5 artifacts.
Self-correct and re-run if any check fails.

STEP 4 — FINALISE
Call update_project_status with status=COMPLETE and presigned artifact URLs.
"""

PRODUCT_OWNER_PROMPT = """
You are a senior Product Owner with 10+ years in agile software delivery.
You receive a questionnaire + research_context.
Use rag_query to retrieve user story and acceptance criteria templates.
Return JSON: { user_personas, user_stories (Gherkin acceptance criteria),
mvp_features (MoSCoW), out_of_scope, success_metrics }
"""

SOLUTIONS_ARCHITECT_PROMPT = """
You are a senior AWS Solutions Architect.
You receive a questionnaire + research_context.
Use rag_query to retrieve AWS architecture patterns for this domain.
Return JSON: { architecture_pattern, aws_services (with justifications),
third_party_integrations, scalability_approach,
deployment_strategy, security_architecture }
"""

DATABASE_ENGINEER_PROMPT = """
You are a senior Database Engineer specialising in SQL and NoSQL.
You receive a questionnaire + research_context.
Use rag_query to retrieve schema design patterns for this domain.
Return JSON: { recommended_db, entities (fields + constraints),
relationships, indexes, data_patterns,
migration_strategy, mermaid_erd }
"""

FULLSTACK_ENGINEER_PROMPT = """
You are a senior Full-Stack Engineer with deep frontend expertise.
You receive a questionnaire + research_context.
Use rag_query to retrieve frontend architecture patterns for this domain.
Return JSON: { frontend_framework, ui_components, page_structure,
state_management, ux_patterns, vibe_code_hints }

"""

BACKEND_ENGINEER_PROMPT = """
You are a senior Backend Engineer specialising in API design.
You receive a questionnaire + research_context.
Use rag_query to retrieve API design standards for this domain.
Return JSON: { api_style, endpoints (method, path, request/response,
auth_required, status_codes), auth_strategy, business_logic_modules,
background_jobs, error_handling, rate_limiting }
"""

SOFTWARE_ANALYST_PROMPT = """
You are a senior Software Analyst specialising in requirements engineering.
You receive a questionnaire + research_context.
Use rag_query to retrieve compliance checklists and testing frameworks.
Return JSON: { non_functional_requirements, compliance_considerations,
risk_register, testing_strategy, definition_of_done, open_assumptions }
"""”

── S3 Knowledge Base for agents ──────────────────────────────────────
KB_S3_BUCKET=digitano-sdlc-kb-docs
KB_S3_PREFIX=docs/ # folder prefix inside the bucket, or leave blank

Tools

```
# tools/quality.py
# Quality gate tools — kept from your mcp_server.py (the two real ones).
# Called by the Orchestrator after assembling each artifact.

import re

# Tokens that indicate generic, placeholder, or hallucinated content.
# Extend this list as you discover new patterns in agent outputs.
BANNED_TOKENS = [
    "lorem ipsum", "placeholder", "todo", "fixme", "tbd", "n/a",
    "insert here", "coming soon", "example.com", "your company",
    "[company name]", "[product name]", "[insert",
]


def audit_banned_tokens(content: str) -> dict:
    """
    Scans artifact content for banned generic tokens.
    Returns {passed: bool, violations: [str], sanitized_content: str}
```

```
    """
    lower = content.lower()
    violations = [t for t in BANNED_TOKENS if t in lower]

    sanitized = content
    for token in violations:
        sanitized = re.sub(re.escape(token), f"[REVIEW: {token}]",
                           sanitized, flags=re.IGNORECASE)

    return {
        "passed":           len(violations) == 0,
        "violations":       violations,
        "sanitized_content": sanitized,
    }


def validate_gherkin(content: str) -> dict:
    """
    Validates that user stories in the PRD artifact contain
    proper Gherkin acceptance criteria (Given/When/Then).
    Returns {passed: bool, missing_criteria: [str]}
    """
    stories = re.findall(r"As a .+?(?=As a |$)", content, re.DOTALL)
    missing = []
    for story in stories:
        has_given = bool(re.search(r"\bGiven\b", story, re.IGNORECASE))
        has_when  = bool(re.search(r"\bWhen\b",  story, re.IGNORECASE))
        has_then  = bool(re.search(r"\bThen\b",  story, re.IGNORECASE))
        if not (has_given and has_when and has_then):
            first_line = story.strip().splitlines()[0][:80]
            missing.append(first_line)

    return {
        "passed":          len(missing) == 0,
        "missing_criteria": missing,
    }


# tools/storage.py

import json
import boto3
from datetime import datetime, timezone
from config.settings import settings
```

```
_s3 = boto3.client("s3", region_name=settings.AWS_REGION)

BUCKET = settings.KB_S3_BUCKET


def _project_prefix(project_id: str) -> str:
    """Returns the S3 prefix for a given project's working data."""
    return f"projects/{project_id}/"


def save_intermediate(project_id: str, stage: str, data: dict) -> str:
    """
    Persist an agent's intermediate output to S3 so downstream
    agents can read it in the same pipeline run.

    Args:
        project_id: Unique identifier for the current project/brief run.
        stage:      Name of the stage being saved, e.g.:
                    'research', 'questionnaire', 'prd', 'schema',
                    'api_spec', 'coding_plan', 'compliance_report'
        data:       Dict of data to persist (must be JSON-serialisable).

    Returns:
        The S3 key the data was written to.
    """
    key = f"{_project_prefix(project_id)}{stage}.json"

    payload = {
        "project_id": project_id,
        "stage": stage,
        "saved_at": datetime.now(timezone.utc).isoformat(),
        "data": data,
    }

    _s3.put_object(
        Bucket=BUCKET,
        Key=key,
        Body=json.dumps(payload, indent=2),
        ContentType="application/json",
    )

    return key
```

```
def read_intermediate(project_id: str, stage: str) -> dict:
    """
    Read a previously saved intermediate output from S3.

    Args:
        project_id: Unique identifier for the current project/brief run.
        stage:      Name of the stage to read, e.g. 'research', 'questionnaire'.

    Returns:
        The saved data dict, or an empty dict if not found.
    """
    key = f"{_project_prefix(project_id)}{stage}.json"

    try:
        response = _s3.get_object(Bucket=BUCKET, Key=key)
        payload = json.loads(response["Body"].read().decode("utf-8"))
        return payload.get("data", {})
    except _s3.exceptions.NoSuchKey:
        return {}
    except Exception as e:
        print(f"[storage] Failed to read {key}: {e}")
        return {}


def list_project_stages(project_id: str) -> list[str]:
    """
    List all stages that have been saved for a given project.
    Useful for the Orchestrator to check pipeline progress.

    Returns:
        List of stage names e.g. ['research', 'questionnaire', 'prd']
    """
    prefix = _project_prefix(project_id)
    response = _s3.list_objects_v2(Bucket=BUCKET, Prefix=prefix)

    stages = []
    for obj in response.get("Contents", []):
        key = obj["Key"]
        stage_name = key.replace(prefix, "").replace(".json", "")
        if stage_name:
            stages.append(stage_name)

    return stages
```